

Operator Reproduction SOP

Supporting Material

Operator SOP: reproduce and film the payment desk

This SOP is for a **disposable devnet demonstration**. It is not custody guidance for production funds.

1. Prepare a clean, observable demo

1. Build the three WASM components with Rust `1.96.1` and install each `manifest.toml` and generated `.wasm` into a disposable ZeroClaw config directory. The per-plugin commands are in the source READMEs.
2. Configure a real ZeroClaw channel, such as Telegram, to route to an agent allowed to use these three tools. Keep the channel credential in ZeroClaw's secret/config storage—not in this template or any terminal recording.
3. Create a new devnet keypair and mint. Fund it only with faucet funds. The build component gets its **public** sender key; signing remains in a separate wallet/CLI process that is never connected to ZeroClaw.
4. Populate the three plugin entries from `redacted-config.toml`, replacing only public placeholders. Restrict mint and recipient allowlists to the disposable values. Use `min_commitment = "finalized"` for confirmation.
5. Confirm discovery before recording:

```
bash zeroclaw plugin list zeroclaw plugin info solana-pay-request zeroclaw plugin info spl-transfer-build zeroclaw plugin info solana-pay-confirm
```

Verify that request declares only `ConfigRead`, and build/confirm declare only `HttpClient`, `ConfigRead`.

2. Prove the live path before filming

1. In the real channel, create a fresh invoice ID and request a small devnet payment. Save the public Solana Pay URL and derived reference.
2. Render the URL locally, outside the agent context:

```
bash ./demo/qr.sh '<SOLANA_PAY_URL>'
```

3. Scan from the phone's disposable devnet wallet. Inspect the recipient, mint, and amount in the wallet, then sign and submit there.
4. Wait for finalization. Independently inspect the public signature with a Solana explorer or a separate public RPC/CLI query.
5. Ask the agent to confirm using only recipient, amount, mint/alias, and invoice ID. Record the `paid:true`, signature, `received_ui`, and `match_count` result. Repeat with an amount one base unit/UI decimal unit different and require `paid:false` before proceeding.
6. If you have time for an extended recording, independently run the build tool against the disposable sender/mint. Its result must say `UNSIGNED`; inspect the resulting bytes outside ZeroClaw. Do not use the

phone wallet's recovery phrase, key export, or signing interface in the agent workflow.

3. Record

Follow `../video/VIDEO_SCRIPT.md` as one continuous 90-second recording. Capture Telegram and the mirrored phone together wherever the channel or wallet approval matters. It is okay to re-record; it is not okay to replace a failed live check with a mock or a slide.

Before upload, scrub:

- shell history and terminal scrollback containing credentials;
- QR codes or browser tabs for unrelated wallets/accounts;
- Telegram contact names/handles that are not part of the demo;
- any seed phrase, private key, API key, bearer token, or credential-bearing RPC URL;
- video metadata that exposes a private filesystem location or account name.

4. Reproduce code-quality claims

From the workspace root:

```
./demo/reproduce.sh
```

This runs the shared-core tests and, for each component, locked tests, host Clippy, WASM Clippy, and a release WASM build. Use `--fast` only for a quick recording-friendly check; do not represent it as the complete build matrix.

5. Operational boundaries after the demo

- Signing/submission is external; verify the decoded transaction at that boundary.
- Use independent RPC endpoints for production confirmation where possible.
- Treat `paid:false` as an observed verdict and `success:false` as a refusal; neither should trigger delivery automatically without the merchant's policy.
- Implement duplicate notification suppression, delivery, accounting, retries, and daily limits outside these stateless components.
- Discard a recent-blockhash proposal when expired. Use durable-nonce mode only after understanding its separately documented nonce-authority and nonce-consumption semantics.